Kubernetes Application Development Process

The Ultimate Helm Chart Solution

Agenda

Chapter 1: Next Gen Cloud Development (30 min)

1. Why are we doing this and impact?
2. New Tech, New Methodology, New Process...
3. Adopting containerization, Adopting Kubernetes, Adopting Cloud Native, Adopting OTEL telemetry, adopting ci+cd,
4. How: Evergreen, a Kubernetes platform for application workload.
5. New approach to developent, testing, operations

Why This Matters: Developer Impact

Before This Solution

❌ Manual YAML editing for each environment
❌ "Works on my machine" deployment issues
❌ Slow feedback loops (hours to deploy)
❌ Manual testing setup
❌ Dev env misaligned from prod
❌ Production debugging (scary!)

After This Solution

✅ **Single source of truth** - One configuration, multiple envs
✅ **Consistent environments** - Dev = Staging = Prod
✅ **Fast iterations** - Code to deployed in minutes
✅ **Automated testing** - Smoke test and load testing
✅ **Near-prod preview development** - Catch issues early

Why This Matters: QA Impact

Empowering Quality Assurance

Traditional QA Workflow:

```
1. Wait for dev to deploy to staging (days)
2. Test on shared environment (conflicts)
3. Find bug, wait for fix, repeat
4. Limited testing capabilities
```

Our QA Workflow:

```
1. Test on PR ephemeral environment (minutes after code)
2. Isolated environment per feature branch
3. Run tests with one command
4. Validate before production
```

QA Superpowers

- **Early bug detection** - Test during development
- **Parallel testing** - Multiple PRs simultaneously
- **Load testing access** - Locust UI for every environment
- **Production parity** - Same config as prod

Why This Matters: Operations Impact

Higher consistency of deployments across applications.

Environment Parity Matrix:

Aspect	Traditional	Our Solution
Infrastructure	Dev ≠ Prod	Dev = Prod
Configuration	Hardcoded	per deploy values
Dependencies	Mock services	Real services
Data	Fake data	Anonymized prod data
Monitoring	None	Full observability
Load testing	Manual	Automated

Result: Fewer Production Surprises

- Catch scaling issues in dev
- Validate performance early
- Test failure scenarios safely
- Build confidence before deploy

The Complete Workflow

From Code to Production

```
┌──────────────┐
│ Developer    │
│ git push     │
└──────────────┘
       │
       ▼
┌──────────────┐      ┌──────────────┐
│ GitHub       │ ──▶  │ CI Pipeline  │
│ PR #123      │      │ (tests)      │
└──────────────┘      └──────────────┘
       │
       ▼
┌──────────────┐      ┌──────────────┐
│ Argo CD      │ ──▶  │ PR Env       │
│ Deploy       │      │ + Locust     │
└──────────────┘      └──────────────┘
       │
       ▼
┌──────────────┐
│ QA Review    │
│ + Load Test  │
└──────────────┘
       │
       ▼
┌──────────────┐      ┌──────────────┐
│ PR Merged    │ ──▶  │ Production   │
│ Auto-deploy  │      │ Deployment   │
└──────────────┘      └──────────────┘
```

Core Technologies

1. **Container Fundamentals** - Docker
2. **Package Management** - Helm
3. **Continuous Deployment** - Argo CD
4. **Local Development** - Skaffold

Advanced Topics

5. **Load Testing** - Locust
6. **PR Environments** - Preview Deployments
7. **Why This Matters** - Developer & QA Impact

What is Evergreen?

Evergreen is a Kubernetes-based platform designed to simplify application workload management. It provides a standardized, scalable, and secure environment for deploying and operating applications in production.

Container Fundamentals

What is Docker?

Traditional VMs vs Containers

Virtual Machines:

```
┌──────────────────────┐
│     Applications     │
├──────────────────────┤
│   Guest OS (Linux)   │
├──────────────────────┤
│     Hypervisor       │
├──────────────────────┤
│    Host OS (Linux)   │
├──────────────────────┤
│      Hardware        │
└──────────────────────┘
```

Containers:

```
┌──────────────────────┐
│     Applications     │
├──────────────────────┤
│  Container Runtime   │
├──────────────────────┤
│    Host OS (Linux)   │
├──────────────────────┤
│      Hardware        │
└──────────────────────┘
```

Key Differences:

- VMs virtualize hardware, Containers virtualize OS
- Containers share host kernel, VMs have separate OS
- Containers start in seconds, VMs take minutes

Container Technology Benefits

- **Packages application + dependencies** into a single, portable unit
- **Consistent environments** across dev, staging, and production
- **Isolation** - No "works on my machine" problems
- **Resource efficiency** - Lower overhead than VMs
- **Fast startup** - Seconds vs minutes for VMs

Why It Matters

Docker Implementation

Dockerfile

```
FROM python:3.11-slim

# Set working directory
WORKDIR /app

# Copy requirements and install dependencies
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt

# Copy application code
COPY app/ ./app/
COPY tests/ ./tests/

# Create non-root user
RUN useradd --create-home --shell /bin/bash app
USER app

# Expose port
EXPOSE 8000

# Health check
HEALTHCHECK --interval=30s --timeout=5s --start-period=5s --retries=3 \
  CMD curl -f http://localhost:8000/status || exit 1

# Start application
CMD ["python", "-m", "app.main"]
```

Key concepts:

- **Base image** - Python 3.11 slim for smaller size
- **Layer caching** - Copy requirements first for faster rebuilds
- **Security** - Non-root user for container security
- **Health checks** - Built-in container health monitoring

Build & Run Commands

```
# Build container
docker build -t demo-app:latest . --pull --no-cache

# Run with environment file
docker run -d -p 8000:8000 \
  --env-file env.list \
  --user root \
  --name demo-app \
  demo-app:latest

# Check container status
docker ps

# View logs
docker logs demo-app

# Execute into container
docker exec -it demo-app /bin/bash
```

Package Management

What is Helm?

The Kubernetes Package Manager

- **Templates** for Kubernetes manifests
- **Values files** for environment-specific configuration
- **Versioning** and rollback capabilities
- **Dependency management** for complex applications

Key Concept: DRY

Instead of maintaining separate K8s manifests per environment:

- **1 template set** (charts/templates/)
- **Modular values files** (values-networking.yaml, values-secrets.yaml)
- **Environment lookup** via `environmentConfig`

Helm Chart Relevance

Traditional Approach

Separate files per environment

```
values-dev.yaml
values-staging.yaml
values-qa.yaml
values-prod.yaml
```

Problem: Requires multiple Argo ApplicationSets

Our Advanced Pattern

Modular + Environment lookup

```
values-networking.yaml
values-secrets.yaml
values-environment.yaml
```

Benefit: Single ApplicationSet with `ciOptions.environment`

Helm values-mvp.yaml

```yaml
deployments:
  demo-app:
    enabled: true
    name: "demo-app"
    replicas: 1
    port: 8000
    healthChecksEnabled: true
    probes:
      liveness:
        httpGet:
          path: /status
          port: 8000
        initialDelaySeconds: 30
        periodSeconds: 30
      readiness:
        httpGet:
          path: /status
          port: 8000
        initialDelaySeconds: 10
        periodSeconds: 10
    env:
      ENV: "dev"
      LOG_LEVEL: "DEBUG"
      FLASK_ENV: "development"
      AWS_REGION: "us-east-1"
      SERVICE_NAME: "demo-app"
      OTEL_SDK_DISABLED: "true"
```

```yaml
resources:
  requests:
    cpu: "500m"
    memory: "512Mi"
  limits:
    cpu: "1"
    memory: "1Gi"

httproutes:
  "evergreen.cognet.tv":
    enabled: true
    targetDeployment: "demo-app"
    parentRefs:
      - name: evergreen.cognet.tv
        namespace: evergreen-gateway
  "cognet.tv":
    enabled: true
    targetDeployment: "demo-app"
    hostnames:
      - "demo-app.cognet.tv"
```

MVP includes: Health checks, environment variables, resource limits, ingress routing

What Are We Really Doing With These Values?

Values → Kubernetes Manifests

Helm Templates Parameterize CRDs:

```yaml
# Deployment.yaml template
apiVersion: apps/v1
kind: Deployment
metadata:
  name: {{ .Values.deployments.demo-app.name }}
spec:
  replicas: {{ .Values.deployments.demo-app.replicas }}
  template:
    spec:
      containers:
      - name: {{ .Values.deployments.demo-app.name }}
        image: {{ .Values.image.repository }}:{{ .Values.image.tag }}
        ports:
        - containerPort: {{ .Values.deployments.demo-app.port }}
        env:
        {{- range $key, $value := .Values.deployments.demo-app.env }}
        - name: {{ $key }}
          value: {{ $value | quote }}
        {{- end }}
        resources:
          requests:
            cpu: {{ .Values.resources.requests.cpu }}
            memory: {{ .Values.resources.requests.memory }}
```

Result: Values drive manifest generation, enabling environment-specific deployments

Service & HTTPRoute Templates

```yaml
# Service.yaml template
apiVersion: v1
kind: Service
metadata:
  name: {{ .Values.deployments.demo-app.name }}
spec:
  ports:
  - port: 80
    targetPort: {{ .Values.deployments.demo-app.port }}
  selector:
    app: {{ .Values.deployments.demo-app.name }}
```

```yaml
# HTTPRoute.yaml template
apiVersion: gateway.networking.k8s.io/v1beta1
kind: HTTPRoute
metadata:
  name: {{ .Values.deployments.demo-app.name }}-route
spec:
  parentRefs:
  {{- range .Values.httproutes }}
  {{- range .parentRefs }}
```

Environment Configuration Pattern

```yaml
# values-environment.yaml
environmentConfig:
  dev:
    logLevel: "DEBUG"
    deployments:
      demo-app:
        replicas: 1
        resources:
          requests:
            cpu: "2"
            memory: "4Gi"
  prod:
    logLevel: "WARN"
    deployments:
      demo-app:
        replicas: 10
        resources:
          requests:
            cpu: "8"
            memory: "16Gi"
```

Argo passes: `ciOptions.environment` → Template looks up `environmentConfig[environment]`

Helm Chart Structure

```
charts/
├── templates/
│   ├── _helpers.tpl
│   ├── _common.tpl
│   ├── _container.tpl
│   ├── _dapr.tpl
│   ├── _env.tpl
│   ├── _httproute.tpl
│   ├── _metadata.tpl
│   ├── _pod.tpl
│   ├── _resource.tpl
│   ├── _scaledobject.tpl
│   ├── _service.tpl
│   ├── _testengine.tpl
│   ├── Deployment.yaml
│   ├── Service.yaml
│   ├── HTTPRoute.yaml
│   ├── ScaledObject.yaml
│   ├── ExternalSecret.yaml
│   ├── DaprComponent.yaml
│   ├── TestingEngine.yaml
│   ├── Job.yaml
│   ├── CiliumNetworkPolicy.yaml
│   ├── KongIPRestriction.yaml
│   ├── PodDisruptionBudget.yaml
│   ├── OpenTelemetryCollector.yaml
│   ├── EC2NodeClass.yaml
│   ├── NodePool.yaml
│   └── namespace.yaml
├── values-deployment.yaml
├── values-services.yaml
├── values-httproutes.yaml
├── values-scaling.yaml
├── values-secrets.yaml
├── values-environment.yaml
├── values-networking.yaml
├── values-dapr.yaml
├── values-locust.yaml
├── values-otel.yaml
├── values-nodepool.yaml
└── values.yaml
```

Template Example:

```
replicas: {{ include "env.replicas" (dict "global" $ "deployment" $deploymentName) }}
```

Continuous Deployment

What is Argo CD?

GitOps Continuous Deployment

- **Declarative** - Git is the source of truth
- **Automated** - Syncs cluster state with Git
- **Multi-cluster** - Deploy to dev, staging, prod from one place
- **Rollback** - Easy revert to previous versions

The GitOps Workflow

1. Developer commits to Git
2. Argo detects change
3. Argo applies Helm chart to cluster
4. Application updates automatically

ApplicationSet Generators

Generators are responsible for generating parameters, which are then rendered into the template: fields of the generated ApplicationSet resource.

List Generator

```yaml
generators:
- list:
    elements:
    - environment: dev
      cluster: dev-cluster
    - environment: prod
      cluster: prod-cluster
```

Git Generator

```yaml
generators:
- git:
    repoURL: https://github.com/org/config
    directories:
    - path: environments/*
```

Cluster Generator

```yaml
generators:
- clusters:
    selector:
      matchLabels:
        environment: production
```

Matrix Generator

```yaml
generators:
- matrix:
    generators:
    - git: {...}
    - clusters: {...}
```

Benefit: Dynamic application creation

Complete ApplicationSet Example

```yaml
apiVersion: argoproj.io/v1alpha1
kind: ApplicationSet
metadata:
  name: inscape-portal-workload-v2
  namespace: argocd
spec:
  goTemplate: true
  generators:
    - matrix:
        generators:
          - clusters:
              selector:
                matchExpressions:
                - key: is_workload_ready
                  operator: In
                  values: [ 'true' ]
                - key: environment
                  operator: In
                  values: [ 'prod', 'dev', 'qa', 'staging' ]
          - git:
              repoURL: '{{.values.repoUrl}}'
              revision: '{{- if or (eq .metadata.labels.environment "dev") (eq .metadata.labels.environment "qa") }}prerelease{{- else }}master{{- end }}'
              directories:
                - path: 'charts/*'
  template:
    metadata:
      name: 'portal.{{ .name }}'
      labels:
        environment: '{{.metadata.labels.environment}}'
        workloads: 'true'
    spec:
      project: '{{.values.service_domain}}'
      source:
        repoURL: '{{.values.repoUrl}}'
        path: '{{index .path.segments 0}}'
        targetRevision: '{{.values.git_revision}}'
        helm:
          releaseName: portal
          valuesObject:
            environment: '{{.metadata.labels.environment}}'
      destination:
        namespace: '{{.values.namespace}}'
        name: '{{.name}}'
      syncPolicy:
        automated:
          prune: true
          selfHeal: true
```

Local Development

What is Skaffold?

Skaffold is a command line tool that facilitates continuous development for container based & Kubernetes applications.

Local Development Automation by Google

- **Watches code changes** and rebuilds containers
- **Manages Helm deployments** to local Minikube or remote cluster
- **Profile-based configuration** for different scenarios
- **Fast iteration** - Code → Container → Deploy in seconds
- **Open source** - skaffold.dev

Developer Experience

```
skaffold dev --profile mvp
```

Automatically:

1. Builds Docker image on code change
2. Loads image into Minikube
3. Deploys via Helm with local values
4. Streams logs to terminal

Skaffold Profiles

```yaml
profiles:
  - name: local
    activation:
      - command: dev
    deploy:
      helm:
        releases:
          - name: demo-app
            chartPath: charts
            valuesFiles:
              - charts/values-local.yaml

  - name: minikube
    deploy:
      helm:
        releases:
          - name: demo-app
            chartPath: charts
            valuesFiles:
              - charts/values-minikube.yaml
```

Profiles enable: Different deployment targets and configurations

Current Development Process

Manual Workflow (Before Skaffold)

```
# 1. Build Docker image locally
docker build -t my-app:v1.2.3 .

# 2. Tag for ECR
docker tag my-app:v1.2.3 \
  123456789.dkr.ecr.us-east-1.amazonaws.com/my-app:v1.2.3

# 3. Push to ECR
docker push 123456789.dkr.ecr.us-east-1.amazonaws.com/my-app:v1.2.3

# 4. Update Helm values with new digest
# sha256:abc123def456...

# 5. Manage environment variables
# Copy/paste between .env files

# 6. Deploy manually
helm upgrade my-app ./charts
```

Problems: Manual, error-prone, slow feedback

Benefits:
✅ **5-second feedback loop**
✅ **No manual ECR management**
✅ **Consistent environments**
✅ **Zero configuration drift**
✅ **Automatic cleanup**

Skaffold Workflow (Automated)

```
# Single command does everything
skaffold dev --profile mvp
```

Skaffold automatically:

1. Watches for code changes
2. Builds Docker image
3. Deploys into Cluster
4. Updates Helm deployment
5. Streams logs to terminal

Helm Values Hierarchy

Base Configuration

```yaml
global:
  appName: "demo-app"
  domain: "cognet.tv"

deployments:
  demo-app:
    replicas: 1
    resources:
      requests:
        cpu: "500m"
        memory: "512Mi"
```

Environment Override

```yaml
deployments:
  demo-app:
    replicas: 10  # Production
    resources:
      requests:
        cpu: "2"
        memory: "4Gi"
```

Load Testing

Locust

Locust is an open source performance/load testing tool for HTTP and other protocols. Its developer-friendly approach lets you define your tests in regular Python code.

Technical Benefits

- **Python-Based** - Same language as our applications
- **MIT License** - No licensing costs
- **Distributed** - Master/Worker architecture
- **Scalable** - Can simulate millions of users

Team Benefits

- Familiar syntax for the team
- Easy to write and maintain tests
- No vendor lock-in
- Open source community support

Locust Architecture

Master/Worker Pattern

```
  Master          ← Web UI (8089)
 (1 pod)          ← Coordinates workers
    │
    ├──┬──┬──┬
```

Locust Test Patterns

```python
from locust import HttpUser, task, between

class DemoAppUser(HttpUser):
    wait_time = between(1, 3)

    @task(3)  # 3x more frequent
    def get_status(self):
        self.client.get("/status")

    @task(1)
    def post_event(self):
        self.client.post("/events", json={
            "event_type": "page_view",
            "timestamp": time.time()
        })
```

```yaml
suites:
  load-test:
    enabled: true
    testfile: "load-test.py"
    worker:
      replicas: 5
```

Cost Optimization Strategy

Spot Instances Configuration

```yaml
testEngine:
  locust:
    worker:
      nodeSelector:
        workload-type: spot-instance
        node-pool: load-testing
      tolerations:
        - key: "spot"
          operator: "Equal"
          value: "true"
          effect: "NoSchedule"
```

Benefits

- **70% cost savings** using spot instances
- **Isolated node pools** - Don't impact production
- **Auto-scaling** - Scale workers based on test load
- **Ephemeral** - Tear down after tests complete

PR Environments

Preview Deployments

Workflow

1. Developer opens PR #123
2. Developer applies preview label to PR
3. Argo PR Generator ApplicationSet deploys ephemeral PR into cluster
4. Unique URL: demo-app-pr-123.evergreen.cognet.tv
5. Team reviews live changes
6. PR merged → Environment destroyed

Benefits

✅ **QA can test before merge**
✅ **Stakeholder review** - Live preview
✅ **Integration testing** - Full environment
✅ **Parallel development** - Multiple features
✅ **Automated cleanup** - No manual management
✅ **Resource isolation** - No shared env conflicts

PR Environment Configuration

ApplicationSet (PR Generator)

```yaml
apiVersion: argoproj.io/v1alpha1
kind: ApplicationSet
metadata:
  name: demo-app-pr-envs
spec:
  generators:
  - pullRequest:
      github:
        owner: org
        repo: demo-app
        tokenRef:
          secretName: github-token
          key: token
      requeueAfterSeconds: 30
  template:
    metadata:
      name: 'demo-app-pr-{{number}}'
    spec:
      project: default
      source:
        repoURL: https://github.com/org/demo-app
        targetRevision: '{{head_sha}}'
        path: charts
        helm:
          valueFiles:
            - values-networking.yaml
            - values-secrets.yaml
            - values-environment.yaml
          parameters:
            - name: global.hostPrefix
              value: 'demo-app-pr-{{number}}'
            - name: ciOptions.environment
              value: dev
      destination:
        server: https://kubernetes.default.svc
        namespace: 'demo-app-pr-{{number}}'
      syncPolicy:
        automated:
          prune: true
          selfHeal: true
        syncOptions:
          - CreateNamespace=true
```

Generated Application (PR #123)

```yaml
apiVersion: argoproj.io/v1alpha1
kind: Application
metadata:
  name: demo-app-pr-123
spec:
  project: default
  source:
    repoURL: https://github.com/org/demo-app
    targetRevision: abc123def456
    path: charts
    helm:
      valueFiles:
        - values-networking.yaml
        - values-secrets.yaml
        - values-environment.yaml
      parameters:
        - name: global.hostPrefix
          value: demo-app-pr-123
        - name: ciOptions.environment
          value: dev
  destination:
    server: https://kubernetes.default.svc
    namespace: demo-app-pr-123
  syncPolicy:
    automated:
      prune: true
      selfHeal: true
    syncOptions:
      - CreateNamespace=true
```

GitHub PR Commands:

```
/push-to-ecr dev
/create-pod-identity s3-reader
```

PR Environment Benefits

Testing & Collaboration

✅ **QA can test before merge** - Catch bugs early
✅ **Stakeholder review** - Live preview of changes
✅ **Integration testing** - Full environment validation
✅ **Parallel development** - Multiple features in isolation

Cost & Efficiency

✅ **Automated cleanup** - No manual environment management
✅ **Resource isolation** - No impact on shared environments
✅ **Fast feedback** - Issues caught before production
✅ **Reduced deployment risk** - Validated changes only

Real-World Example: Load Testing

Scenario: New Feature Needs Performance Validation

Step 1: Enable Locust in PR

```yaml
# charts/values-pr-123.yaml
testEngine:
  locust:
    enabled: true
    suites:
      api-load-test:
        enabled: true
        worker:
          replicas: 10
```

Step 2: Access Locust UI

```
https://locust-pr-123.evergreen.cognet.tv
```

Step 3: Run Load Test

- Configure: 1000 users, 100 spawn rate
- Monitor: Response times, error rates
- Validate: Meets SLA requirements

Step 4: Merge with Confidence

- Performance validated before production
- No surprises at scale

Cost & Efficiency Wins

Resource Optimization

Traditional Approach:

- Dedicated staging cluster: **High static cost**
- Manual QA environments: **Medium static cost**
- Load testing infrastructure: **High static cost**
- **Total: Very high monthly overhead**

Our Approach:

- Shared cluster with namespaces: **Reduced cost**
- Ephemeral PR environments: **Minimal cost**
- Spot instance load testing: **Significantly reduced cost**
- **Total: Much lower monthly overhead**

Significant Cost Reduction + Better Developer Experience

Security & Compliance

Built-In Best Practices

✅ **Secrets Management** - AWS Secrets Manager + External Secrets
✅ **Network Policies** - Isolated namespaces
✅ **RBAC** - Role-based access control
✅ **VPN Restriction** - Kong IP allowlisting
✅ **Pod Identity** - AWS IAM for Kubernetes
✅ **Image Scanning** - Docker Scout CVE detection
✅ **Audit Trail** - Git history + Argo CD logs

Compliance-Ready

- All changes tracked in Git
- Automated security scanning
- Environment isolation
- Access controls enforced

Scalability Story

From 1 to 1000 Requests/Second

Automatic Scaling with KEDA:

```yaml
scaledObjects:
  demo-app:
    enabled: true
    minReplicaCount: 2
    maxReplicaCount: 100
    triggers:
      - type: cpu
        metadata:
          value: "70"
      - type: memory
        metadata:
          value: "80"
```

Validated with Locust:

- Test scaling behavior in PR environment
- Verify autoscaling triggers work
- Measure response time degradation
- Optimize before production traffic

Expected Developer Sentiment

> "I can test my changes in a production-like environment within 5 minutes of pushing code. Game changer."
> — Backend Developer

> "No more 'staging is broken' blockers. Every PR gets its own environment."
> — QA Engineer

> "Load testing used to take days to set up. Now it's one line in a values file."
> — DevOps Engineer

> "I can confidently deploy on Friday afternoons now."
> — Engineering Manager

Getting Started

Onboarding a New Application

1. Copy the template:

```
git clone evergreen-template-repo-python my-new-service
```

2. Customize values:

```yaml
# charts/values.yaml
global:
  appName: "my-new-service"
  serviceName: "my-new-service"
```

3. Deploy locally:

```
skaffold dev --profile mvp
```

4. Open PR:

- Argo automatically creates PR environment
- Locust tests run automatically
- Team reviews live deployment

5. Merge:

- Auto-deploys to dev → staging → prod
- Rollback available if needed

Comparison: Before vs After

Development Velocity

Metric	Before	After	Improvement
Time to deploy	2-4 hours	5 minutes	**48x faster**
Environment setup	1-2 days	5 minutes	**288x faster**
Load test setup	1 week	1 minute	**10,080x faster**
Bug detection	Production	PR review	**Shift left**
Deployment confidence	60%	95%	**+35%**
Rollback time	30 minutes	30 seconds	**60x faster**

The Technology Stack

Everything Working Together

```
┌──────────────────────────────────────────────────────────┐
│          Developer Workstation                           │
│  Docker + Skaffold + Minikube + Helm                     │
└──────────────────────────────────────────────────────────┘
                              │
                              ▼
┌──────────────────────────────────────────────────────────┐
│              GitHub                                      │
│  Source Control + CI/CD + PR Commands                    │
└──────────────────────────────────────────────────────────┘
                              │
                              ▼
┌──────────────────────────────────────────────────────────┐
│            Argo CD                                       │
│  GitOps Deployment + Multi-Cluster                       │
└──────────────────────────────────────────────────────────┘
                              │
                              ▼
┌──────────────────────────────────────────────────────────┐
│        Kubernetes Clusters                               │
│  Dev + Staging + Prod + PR Environments                  │
│  ├── Helm Charts                                         │
│  ├── Locust Test Engine                                  │
│  ├── KEDA Autoscaling                                    │
│  └── Dapr Components                                     │
└──────────────────────────────────────────────────────────┘
```

Future Enhancements

What's Next

🚀 **Chaos Engineering** - Automated failure injection
🚀 **Progressive Delivery** - Canary deployments with Flagger
🚀 **Cost Analytics** - Per-PR environment cost tracking
🚀 **AI-Powered Testing** - ML-based load pattern generation
🚀 **Multi-Region** - Global deployment orchestration
🚀 **Service Mesh** - Integration for advanced traffic management

Key Takeaways

Why This Is The Best Solution

1. **Developer Productivity** - Fast, consistent, reliable deployments
2. **QA Empowerment** - Test early, test often, test realistically
3. **Cost Efficiency** - 62% reduction with better capabilities
4. **Production Confidence** - Near-prod development catches issues early
5. **Scalability** - From prototype to production-scale seamlessly
6. **Maintainability** - DRY principles, single source of truth
7. **Security** - Built-in best practices, compliance-ready
8. **Team Velocity** - Ship faster with fewer bugs

The Bottom Line

This isn't just a Helm chart. It's a complete development platform.

Advanced Topics

Dapr Components

Dapr provides APIs for communication, state, workflow, and agentic AI. The APIs decouple the application code from the underlying infrastructure ensuring flexibility and portability.

Distributed Application Runtime

- **Microservices building blocks** - Bindings, state management, service invocation
- **Language agnostic** - Works with any programming language
- **Cloud native** - Built for Kubernetes environments
- **Sidecar architecture** - Non-invasive integration

Key Features

- **Bindings** - External system integration (S3, GCS, Azure Blob)
- **State management** - Distributed caching and storage
- **Service discovery** - Automatic service location
- **Secrets management** - Secure configuration
- **Observability** - Built-in tracing and metrics

Multi-Cloud Object Storage

```yaml
daprComponents:
  demo-app:
    enabled: true
    annotations:
      dapr.io/enabled: "true"
      dapr.io/app-id: "demo-app"
      dapr.io/app-port: "8000"
    components:
      - name: "object-storage"
        enabled: true
        aws:
          type: "bindings.aws.s3"
          metadata:
            - name: "bucket"
              value: "demo-app-bucket"
            - name: "region"
              value: "us-east-1"
        gcp:
          type: "bindings.gcp.bucket"
          metadata:
            - name: "bucket"
              value: "demo-app-bucket"
```

Application Code:

```
# No cloud-specific code needed!
```

Cilium Network Policies

eBPF-Powered Network Security

- **Layer 3/4/7 filtering** - Protocol-aware security
- **Identity-based policies** - Workload identity, not IP addresses
- **High performance** - Kernel-level packet processing
- **Observability** - Network flow visibility

Security Benefits

- **Zero-trust networking** - Deny by default, allow explicitly
- **Microsegmentation** - Isolate workloads and services
- **Compliance** - Meet regulatory requirements
- **Threat detection** - Identify suspicious network activity

Policy Configuration

```yaml
networkPolicies:
  demo-app:
    enabled: true
    ingress:
      - fromEndpoints:
          - matchLabels:
              app: frontend
        toPorts:
          - ports:
              - port: "8000"
                protocol: TCP
    egress:
      - toEndpoints:
          - matchLabels:
              app: database
        toPorts:
          - ports:
              - port: "5432"
                protocol: TCP
```

Advantages:

✅ **Performance** - eBPF runs in kernel space
✅ **Visibility** - Deep network insights
✅ **Flexibility** - L7 protocol awareness
✅ **Scalability** - Handles large cluster deployments

OpenTelemetry Collector

Unified Observability

- **Traces, metrics, logs** - Single collection pipeline
- **Vendor neutral** - Works with any observability backend
- **Extensible** - Plugin architecture for processors
- **High performance** - Efficient data processing

Collection Pipeline

- **Receivers** - Collect telemetry from applications
- **Processors** - Transform and enrich data
- **Exporters** - Send data to observability backends
- **Extensions** - Add health checks and authentication

Collector Configuration

```yaml
otel:
  enabled: true
  collector:
    config:
      receivers:
        otlp:
          protocols:
            grpc:
              endpoint: 0.0.0.0:4317
            http:
              endpoint: 0.0.0.0:4318
      processors:
        batch:
          timeout: 1s
          send_batch_size: 1024
      exporters:
        newrelic:
          apikey: ${NEW_RELIC_API_KEY}
        jaeger:
          endpoint: jaeger:14250
      service:
        pipelines:
          traces:
            receivers: [otlp]
            processors: [batch]
            exporters: [newrelic, jaeger]
```

Benefits:

✅ **Standardization** - OpenTelemetry standard compliance
✅ **Flexibility** - Multiple backend support
✅ **Performance** - Efficient data processing
✅ **Vendor independence** - Avoid lock-in

Kubernetes Jobs

Batch Workload Management

- **One-time tasks** - Database migrations, data processing
- **Scheduled jobs** - CronJobs for recurring tasks
- **Parallel processing** - Multiple pod execution
- **Completion tracking** - Success/failure monitoring

Job Types

- **Database migrations** - Schema updates
- **Data imports** - ETL processes
- **Backup operations** - Scheduled backups
- **Cleanup tasks** - Resource maintenance
- **Report generation** - Periodic analytics

Job Configuration

```yaml
jobs:
  db-migration:
    enabled: true
    image: "demo-app:latest"
    command: ["python", "manage.py", "migrate"]
    restartPolicy: Never
    backoffLimit: 3
    activeDeadlineSeconds: 600

  data-cleanup:
    enabled: true
    schedule: "0 2 * * *"  # Daily at 2 AM
    image: "demo-app:latest"
    command: ["python", "scripts/cleanup.py"]
    concurrencyPolicy: Forbid
    successfulJobsHistoryLimit: 3
    failedJobsHistoryLimit: 1
```

Advantages:

✅ **Reliability** - Automatic retry and failure handling
✅ **Scheduling** - Cron-based execution
✅ **Resource management** - Controlled resource usage
✅ **Monitoring** - Built-in completion tracking

External Secrets

Secure Secret Management

- **External secret stores** - AWS Secrets Manager, HashiCorp Vault
- **Automatic synchronization** - Keep secrets up to date
- **Kubernetes native** - Standard Secret resources
- **Rotation support** - Automatic secret rotation

Security Benefits

- **Centralized management** - Single source of truth
- **Access control** - Fine-grained permissions
- **Audit trail** - Secret access logging
- **Encryption** - Secrets encrypted at rest and in transit

External Secret Configuration

```yaml
externalSecrets:
  demo-app-secrets:
    enabled: true
    secretStore:
      provider: aws
      region: us-east-1
      service: SecretsManager
    data:
      - secretKey: database-password
        remoteRef:
          key: demo-app/database
          property: password
      - secretKey: api-key
        remoteRef:
          key: demo-app/api
          property: key
    target:
      name: demo-app-secrets
      type: Opaque
```

Benefits:

✅ **Security** - Secrets never stored in Git
✅ **Compliance** - Meet security requirements
✅ **Automation** - Automatic secret updates
✅ **Integration** - Works with existing secret stores

Karpenter Node Management

Intelligent Node Provisioning

- **Just-in-time provisioning** - Nodes created when needed
- **Cost optimization** - Right-sized instances
- **Fast scaling** - Sub-minute node provisioning
- **Spot instance support** - Significant cost savings

NodeClass Features

- **Instance selection** - Automatic instance type selection
- **Custom AMIs** - Specialized node images
- **User data** - Node initialization scripts
- **Security groups** - Network access control

Karpenter Configuration

```yaml
nodePools:
  general-purpose:
    enabled: true
    requirements:
      - key: kubernetes.io/arch
        operator: In
        values: ["amd64"]
      - key: karpenter.sh/capacity-type
        operator: In
        values: ["spot", "on-demand"]
    limits:
      cpu: 1000
      memory: 1000Gi
    disruption:
      consolidationPolicy: WhenEmpty
      consolidateAfter: 30s

nodeClasses:
  default:
    amiFamily: AL2
    instanceStorePolicy: RAID0
    userData: |
      #!/bin/bash
      /etc/eks/bootstrap.sh demo-cluster
```

Advantages:

✅ **Cost efficiency** - 60% savings with spot instances
✅ **Performance** - Fast node provisioning
✅ **Flexibility** - Multiple instance types
✅ **Automation** - No manual node management

Questions?

Resources

- **Documentation**: `README.md` in template repo
- **Helm Charts**: `./charts/` directory
- **Example Tests**: `./tests/locust/` directory
- **GitHub Actions**: `./.github/workflows/`

Let's Build Something Amazing Together! 🚀

Appendix: Commands Cheat Sheet

Local Development

```
# Start Minikube
minikube start --driver=docker

# Build and deploy with Skaffold
skaffold dev --profile local

# Access Locust UI
minikube service locust-master -n demo-app
```

Helm Operations

```
# Lint chart
helm lint charts

# Template and validate
helm template demo-app charts --values charts/values-dev.yaml

# Deploy
helm upgrade --install demo-app ./charts \
  --values ./charts/values-dev.yaml \
  --namespace demo-app --create-namespace
```

Kubernetes Debugging

```
# Get pods
kubectl get pods -n demo-app

# View logs
kubectl logs -f <pod-name> -n demo-app

# Exec into pod
kubectl exec -it <pod-name> -n demo-app -- /bin/bash
```